

Core Java – Chapter 12

Database, MySQL & JDBC — Summary Notes

What is a Database?

A **database** is an organized collection of data stored electronically so it can be easily **accessed, managed, and updated**.

Database in Java — In Java, a database is accessed using **JDBC (Java Database Connectivity)**, which allows Java programs to:

- Store data
- Retrieve data
- Update data
- Delete data

Java itself does **not store data permanently**; databases like MySQL handle permanent storage. Example use cases: student records, banking systems, e-commerce applications, employee management systems.

What is MySQL?

MySQL is:

- An open-source Relational Database Management System (RDBMS).
- Built on **SQL (Structured Query Language)**.
- Stores data in the form of tables (rows and columns).

Why MySQL? Fast and reliable · Easy to learn · Widely used with Java, PHP, Python · Free and open source.

Parts of MySQL (Architecture Components)

Component	Description / Example
1. Database	A container that holds tables. Example: school_db
2. Table	Stores data in rows and columns. Example: students
3. Rows (Records)	Each row represents one entry. Example: one student's data
4. Columns (Fields)	Each column represents an attribute. Example: id, name, marks
5. Primary Key	Uniquely identifies each record; cannot be NULL. Example: student_id
6. Foreign Key	Links two tables and maintains relationships between them

MySQL Operators

Operators are symbols used to perform operations in SQL queries.

1. Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

```
SELECT marks + 10 FROM students;
```

2. Comparison Operators

Operator	Meaning
=	Equal
!= or <>	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

```
SELECT * FROM students WHERE marks > 60;
```

3. Logical Operators

Operator	Meaning
AND	Both conditions true
OR	Any condition true
NOT	Reverses condition

```
SELECT * FROM students WHERE marks > 60 AND age >= 18;
```

MySQL Clauses

Clauses help **filter, sort, and group data**.

Clause	Purpose	Example
WHERE	Filters records	SELECT * FROM students WHERE marks > 50;
SELECT	Retrieves data	SELECT name, marks FROM students;

FROM	Specifies the table	SELECT * FROM students;
ORDER BY	Sorts results	SELECT * FROM students ORDER BY marks DESC;
GROUP BY	Groups rows with the same values	SELECT course, COUNT(*) FROM students GROUP BY course;
HAVING	Filters groups (used with GROUP BY)	SELECT course, COUNT(*) FROM students GROUP BY course HAVING COUNT(*) > 5;

Functions in MySQL

Functions perform calculations and return a value.

1. Aggregate Functions

Function	Use
COUNT()	Number of rows
SUM()	Total value
AVG()	Average
MAX()	Highest value
MIN()	Lowest value

```
SELECT AVG(marks) FROM students;
```

2. String Functions

Function	Use
UPPER()	Convert to uppercase
LOWER()	Convert to lowercase
LENGTH()	String length

```
SELECT UPPER(name) FROM students;
```

3. Date Functions

```
SELECT CURDATE();
SELECT NOW();
```

Procedures in MySQL (Stored Procedures)

A **stored procedure** is a group of SQL statements saved in the database that can be executed repeatedly.

Advantages: faster execution, reusable code, better security, reduces repeated SQL code.

```
DELIMITER //
CREATE PROCEDURE getStudents()
```

```

BEGIN
  SELECT * FROM students;
END //
DELIMITER ;

-- Calling the procedure
CALL getStudents();

```

Relation Between Java and MySQL

Java Side	MySQL Side (Role)
Java program	Application logic
JDBC	Bridge
MySQL	Data storage

Overall flow: Java → JDBC → MySQL → Result → Java

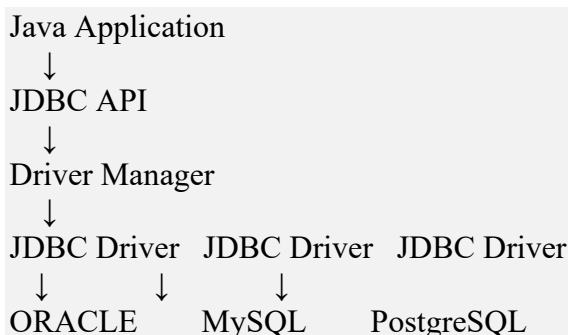
Java Database Connectivity (JDBC)

The JDBC diagram shows how a Java application communicates with a database like MySQL.

Flow of JDBC Architecture

- **1. Java Application** — contains Java code; sends SQL queries (SELECT, INSERT, UPDATE, DELETE).
- **2. JDBC API** — a set of Java classes and interfaces (java.sql, javax.sql) that acts as a bridge between Java and the database.
- **3. JDBC Driver** — converts Java JDBC calls into database-specific instructions. Example: MySQL Connector/J.
- **4. Database (MySQL)** — stores data in tables, executes SQL queries, and sends results back to Java.

JDBC Driver Architecture



(The Driver Manager sits between the JDBC API and three parallel JDBC Drivers, one per database type.)

The Driver Manager selects the correct JDBC driver for whichever database (Oracle, MySQL, PostgreSQL, etc.) the application connects to.

Step-by-Step JDBC Call Sequence

- **0.** Load the database driver → registered with the DriverManager.
- **1.** DriverManager.getConnection() → produces a **Connection** object.
- **2.** connection.createStatement() → produces a **Statement** object.
- **3a.** For SELECT: statement.executeQuery() → produces a **ResultSet**.
- **3b.** For INSERT / UPDATE / DELETE: statement.executeUpdate() → returns an **int** (number of rows affected).
- **4a.** On the ResultSet: (i) next() moves to the next row in a loop, (ii) getXxx() reads the column data.
- **5.** close() is called on the ResultSet, Statement, and Connection to release resources when done.

Java API (Application Programming Interface)

A **Java API** is a collection of ready-made classes, interfaces, and packages provided by Java to help programmers perform common tasks easily — instead of writing everything from scratch, programmers reuse Java API classes.

Why Java API is needed:

Without Java API	With Java API
Programmers must write all code manually	Faster development
Development is slow and complex	Reliable and tested code, fewer errors, code reusability

java.sql (JDBC) — Used for Database Connectivity

Interface / Class	Use
Connection	Represents the database connection
Statement	Executes SQL queries
ResultSet	Stores/holds the results returned by a query

Part C: SQL Command Categories

Abbreviation	Full Form	Purpose
DDL	Data Definition Language	Defines database structure
DML	Data Manipulation Language	Manipulates data
DQL	Data Query Language	Retrieves data
TCL	Transaction Control Language	Controls transactions
DCL	Data Control Language	Controls access

1. DDL – Data Definition Language

Used to define, create, modify, and delete the structure of database objects such as tables, databases, and indexes.

- **CREATE** – creates a new database or table

- **ALTER** – modifies an existing table structure
- **DROP** – deletes a table or database
- **TRUNCATE** – removes all records from a table (structure remains)
- **RENAME** – renames a database object

```
-- Create & use a database
CREATE DATABASE company;
USE company;
```

```
-- Create table
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary INT,
    dept VARCHAR(30)
);
```

```
-- Alter, rename, truncate, drop
ALTER TABLE employees ADD age INT;
RENAME TABLE employees TO staff;
TRUNCATE TABLE staff;
DROP TABLE staff;
```

2. DML – Data Manipulation Language

Used to insert, update, delete, and modify data stored in tables.

- **INSERT** – adds new records to a table
- **UPDATE** – modifies existing records
- **DELETE** – removes records from a table

```
INSERT INTO employees VALUES (1, 'John', 30000, 'IT');
INSERT INTO employees VALUES (2, 'Emma', 35000, 'HR');
```

```
UPDATE employees SET salary = 40000 WHERE emp_id = 1;
```

```
DELETE FROM employees WHERE emp_id = 2;
```

3. DQL – Data Query Language

Used to retrieve data from the database. Common command: **SELECT** – fetches data from one or more tables.

```
SELECT * FROM employees;
SELECT emp_name, salary FROM employees;
SELECT * FROM employees WHERE dept = 'IT';
```

4. TCL – Transaction Control Language

Used to manage transactions and control changes made by DML commands.

- **COMMIT** – saves changes permanently
- **ROLLBACK** – undoes changes
- **SAVEPOINT** – sets a point to roll back to later

```
START TRANSACTION;
INSERT INTO employees VALUES (3, 'Alice', 28000, 'Finance');
SAVEPOINT sp1;
```

```
DELETE FROM employees WHERE emp_id = 3;
ROLLBACK TO sp1;
COMMIT;
```

5. DCL – Data Control Language

Used to control access and permissions in the database.

- **GRANT** – gives permissions to users
- **REVOKE** – removes permissions from users

Part D: Worked Practice Examples

DDL Practice — teacher Table

```
CREATE SCHEMA education;
USE education;
```

```
CREATE TABLE teacher (
    name VARCHAR(10),
    age INT,
    department VARCHAR(20)
);
```

-- Add a new column

```
ALTER TABLE teacher ADD COLUMN degree VARCHAR(100);
```

-- Rename a column

```
ALTER TABLE teacher CHANGE department dept VARCHAR(20);
```

-- Change a column's data type

```
ALTER TABLE teacher CHANGE age age VARCHAR(20);
```

-- Drop the table

```
DROP TABLE teacher;
```

-- Remove all rows but keep the structure

```
TRUNCATE TABLE teacher;
```

TRUNCATE deletes all records, keeps the table structure intact, cannot be rolled back, and is faster than DELETE.

Insert & Update Practice

```
INSERT INTO teacher VALUES ('sri', 24, 'cse', 'B.E');
```

-- 'sri' -> name, 24 -> age, 'cse' -> dept, 'B.E' -> degree

```
SELECT * FROM teacher; -- shows all columns and records
```

-- Updates age for ALL records (no WHERE clause)

```
UPDATE teacher SET age = 23;
```

-- Update a specific person only — always use WHERE

```
UPDATE teacher SET age = 23 WHERE name = 'sri';
```

Forgetting the WHERE clause on an UPDATE or DELETE changes/removes every row in the table — always double-check the condition before running one.

DELETE vs. DROP (DML + DDL)

Command	Example	Effect
DELETE (condition)	DELETE FROM students WHERE department = 'computer';	Deletes only matching rows (here, computer dept.)
DELETE (condition)	DELETE FROM students WHERE age = 30;	Deletes only students aged 30
DELETE (all rows)	DELETE FROM students;	Removes all rows; table structure remains
DROP TABLE	DROP TABLE students;	Deletes the table AND its data permanently

Aggregate Function Practice (students table)

```
SELECT SUM(students_mark) FROM students;  
SELECT SUM(students_mark) AS totalmarks FROM students; -- total marks
```

```
SELECT MAX(students_mark) FROM students; -- highest mark  
SELECT MIN(students_mark) FROM students; -- lowest mark  
SELECT AVG(students_mark) FROM students; -- average marks
```

```
SELECT COUNT(name) FROM students WHERE department = 'cse';  
-- counts students in the CSE department
```

SELECT, ORDER BY, GROUP BY Practice (student / employee tables)

```
SELECT * FROM student WHERE department = 'cse';  
SELECT stud_name, mark FROM student;
```

-- ORDER BY

```
SELECT stud_name, mark FROM student ORDER BY mark DESC;
```

-- Aggregate functions

```
SELECT SUM(mark) FROM student;  
SELECT AVG(mark) FROM student;
```

-- GROUP BY

```
SELECT department, SUM(mark) FROM student GROUP BY department;  
SELECT department, COUNT(name) FROM student GROUP BY department;  
SELECT department, COUNT(name) AS total_students  
FROM student GROUP BY department ORDER BY total_students;
```

-- Employee table

```
SELECT COUNT(*) FROM employee WHERE department = 'IT';  
SELECT department, COUNT(*) FROM employee GROUP BY department;
```

```
SELECT department, AVG(salary) FROM employee GROUP BY department;
```

-- HAVING (filters groups, applied after GROUP BY)

```
SELECT department, COUNT(*) FROM employee
```

```
GROUP BY department
```

```
HAVING AVG(salary) < 60000;
```

Safe Update Mode

MySQL Workbench's Safe Update Mode blocks UPDATE/DELETE statements that don't use a key column in the WHERE clause, to prevent accidental mass changes. Toggle it with:

```
SET SQL_SAFE_UPDATES = 0; -- turn off (allow unsafe updates)
```

```
SET SQL_SAFE_UPDATES = 1; -- turn back on (recommended default)
```

End of Chapter 12 – Database, MySQL & JDBC